

Submission: Week 4 - Three Roads Stack Decision

AI Fluency Track • Architecture Evaluation & Tech Stack Rationale

1. Architectural Strategy & Constraints

Choosing how to build is an essential AI-fluency skill. Rather than letting AI blindly dictate the architecture, this evaluation feeds explicit real-world constraints into the prompt, evaluates trade-offs across three distinct options, pressure-tests the front-runner, and establishes a clear, sustainable stack decision.

Core Constraints Inputted:

- 1. Budget: Strictly 100% Free Tier Hosting & Tools.
- 2. Skill Level: Comfortable with HTML/CSS & Basic Modern JS / AI Assistance; avoiding complex DevOps.
- 3. Portfolio Content: Multi-page layout, embedded interactive demos, code repo links, and long-form case studies.
- 4. Display Needs: Image galleries, syntax-highlighted code blocks, responsive embedded frames.
- 5. Dynamic Requirement: Not yet required (Static site generation with client-side widgets is sufficient).

2. Stack Options Comparison (Simplest to Most Powerful)

Option & Tier	Build Method	Free Host	Backend?	Key Trade-off
Option A: Simplest	No-Code / Static HTML + Tailwind CSS	GitHub Pages / Netlify	No (Static)	Ultra-fast setup, but repetitive HTML updates across pages without a templating engine.
Option B: Balanced [Chosen]	Astro / Next.js Static Export + Tailwind	Vercel / Cloudflare Pages	Not yet (Static SSG)	Slight learning curve for component routing, but modular, scalable, and lightning fast.
Option C: Most Powerful	Full-Stack Next.js (App Router) + Database	Vercel + Supabase Free	Yes (Serverless)	Unmatched dynamic capabilities, but higher maintenance overhead and cold-start

complexity.

3. Pressure-Testing the Front-Runner

- **What breaks if I pick the simplest (Option A)?** Maintaining global layouts (headers, footers, navs) becomes error-prone. Changing a single menu link requires editing every single HTML file manually.
- **What do I maintain if I pick the most powerful (Option C)?** Database schemas, serverless function cold starts, API route security, and potential tier limit monitoring on database providers like Supabase.
- **Can I finish in two weeks?** Yes. Option B allows complete site layout and content integration well within the 14-day timeline by leveraging component-based development and AI code generation.
- **Does it show my work the way it needs to be shown?** Absolutely. Component architecture effortlessly handles responsive image grids, syntax-highlighted code blocks, and embedded iframe demos without extra bloat.

4. Final Decision Rationale (In My Own Words)

Chosen Stack: Astro / Next.js Static Export on Vercel

I chose Option B (Astro/Next.js Static Export on Vercel) after evaluating all three paths against my real constraints. Option A (Pure HTML/CSS) was discarded because managing duplicate headers, navigation, and layout changes across multiple case studies would quickly become a nightmare to maintain. Option C (Full-Stack Next.js with Database) was discarded because I do not need dynamic server features yet; adding a database and backend infrastructure right now introduces unnecessary maintenance overhead and potential failure points without adding real value to my portfolio.

Option B hits the absolute sweet spot. It provides component reusability (so I can maintain this effortlessly) and allows me to write case studies cleanly in Markdown or MDX. Most importantly, it shows my work well—delivering fast load times, beautiful syntax highlighting for code repos, crisp responsive image galleries, and seamless iframe embeds for live demos. On the backend question: no, I do not need a backend yet. A static site deployment hosted 100% free on Vercel is the clean, robust choice I can finish in two weeks and comfortably maintain.

5. Pass / Revise Criteria Checklist

Requirement	Status	Details
Three genuine options evaluated	[PASS]	Simplest (Option A), Balanced (Option B),

		Most Powerful (Option C).
Chosen stack free & matched to display needs	[PASS]	Vercel free tier with native image, iframe, and markdown support.
Rationale includes 'can I maintain this'	[PASS]	Explicitly addresses long-term component maintenance and simplicity.
Backend question answered honestly	[PASS]	Confirmed 'Not yet needed' — avoiding premature complexity.

Assignment Reference: <https://aifluency.flyrank.ai/week-04.html#three-roads>